

Critic-based PPO for Flow-Matching Models

Algorithm, Pseudocode, and Theoretical Analysis (Flow-Factory)

Flow-Factory

June 19, 2026

Abstract

This note documents the critic-based Proximal Policy Optimization (PPO) trainer in Flow-Factory (`trainer_type: ppo`). It is a *coupled* actor-critic algorithm: it reuses the same SDE rollout and per-step log-probability machinery as GRPO, but replaces GRPO’s single, group-normalized advantage (broadcast to every denoising step) with a learned, timestep-conditioned *value critic* and Generalized Advantage Estimation (GAE), giving fine-grained per-step credit assignment. We formalize denoising as a finite-horizon MDP, give the data-flow diagram, pseudocode, and a short theoretical analysis (variance reduction, GAE bias-variance, the clipped surrogate). We close with a detailed comparison to *Explicit Critic Guidance for Aligning Diffusion Models* ([arXiv:2605.27736](https://arxiv.org/abs/2605.27736)), which makes the diffusion backbone its own value function—precisely the “backbone-critic” variant Flow-Factory evaluated and deferred.

Contents

1	Introduction and Motivation	2
2	Preliminaries: Denoising as an MDP	2
3	From GRPO to Critic-based PPO	2
4	Algorithm and Data Flow	3
4.1	Value critic	3
4.2	Generalized Advantage Estimation	4
4.3	PPO objective	4
4.4	Pseudocode	5
4.5	Decoupling critic and policy update frequency	6
4.6	Periodic critic re-warmup (resample)	7
5	Theoretical Analysis	8
5.1	State-dependent baselines are unbiased and reduce variance	8
5.2	GAE bias-variance	8
5.3	Trust-region surrogate	8
5.4	Train-inference consistency	8
6	Comparison with <i>Explicit Critic Guidance</i> (arXiv:2605.27736)	8
7	Implementation Notes and Configuration	10
8	Limitations and Future Work	10

1 Introduction and Motivation

Flow-Factory fine-tunes flow-matching / diffusion models with online RL against (typically non-differentiable) reward models. The default algorithm, GRPO, treats a whole denoising rollout as a single “action”: it computes one scalar advantage per sample from *group-relative* reward normalization and broadcasts it to all training timesteps. This is simple and strong, but has two structural limitations:

1. **No per-step credit.** Every denoising step in a trajectory shares the same advantage, so the gradient cannot distinguish which steps were responsible for a high/low reward.
2. **Group-mean baseline.** The variance-reduction baseline is the per-group reward mean; it ignores the *state* (the noisy latent), and its quality depends on the group size.

Critic-based PPO addresses both by learning a state-conditioned value function $V_\phi(\mathbf{z}_s, t_s)$ over noisy latent states and forming *per-step* advantages with GAE. The contribution of this implementation is a *lightweight, model-agnostic* actor-critic that drops into the existing rollout pipeline:

- an independent value critic that consumes the full generation latent at any layout (conv / packed / video) and is built eagerly from the channel count (Section 4.1);
- GAE per-step advantages with optional global whitening (Section 4.2);
- a PPO objective with policy-ratio clipping and value clipping, a critic warmup phase, and two optimizers sharing one backward (Section 4.3).

2 Preliminaries: Denoising as an MDP

SDE sampling. A rollout produces a trajectory of latents $\mathbf{z}_0 \rightarrow \mathbf{z}_1 \rightarrow \dots \rightarrow \mathbf{z}_S$, where $\mathbf{z}_0$ is initial noise and $\mathbf{z}_S$ decodes to the generated image/video. At each stochastic (SDE) step s the scheduler (Flow-SDE) defines a Gaussian transition (the *policy*)

$$\pi_\theta(\mathbf{z}_{s+1} \mid \mathbf{z}_s, t_s) = \mathcal{N}(\mathbf{z}_{s+1}; \boldsymbol{\mu}_\theta(\mathbf{z}_s, t_s), \sigma_s^2 \mathbf{I}), \quad \sigma_s = \text{std_dev_t} \cdot \sqrt{-\text{dt}}, \quad (1)$$

whose mean $\boldsymbol{\mu}_\theta$ is a deterministic function of the network’s velocity prediction. The per-step log-probability $\log \pi_\theta$ is tractable and is exactly what the scheduler returns and stores during rollout, which is the foundation of train–inference consistency.

MDP. We treat the ordered SDE steps as a finite-horizon MDP:

state	$s_s = (\mathbf{z}_s, t_s)$ (noisy latent + scheduler timestep)
action	$a_s = \mathbf{z}_{s+1}$ (the sampled next latent)
policy	$\pi_\theta(a_s \mid s_s)$ from Eq. (1)
reward	$r_s = R \cdot \mathbb{I}[s = S-1]$, with terminal $R = \sum_k w_k r_k(\text{decode}(\mathbf{z}_S))$
value	$V_\phi(\mathbf{z}_s, t_s)$, terminal $V(\mathbf{z}_S) \equiv 0$

The reward is *terminal only*: it is the (weighted) reward-model score of the final decoded sample, received on the last SDE transition. Figure 1 illustrates the episode.

3 From GRPO to Critic-based PPO

GRPO. For a group g of K rollouts sharing a prompt, GRPO assigns each sample the same advantage at *every* step:

$$A_i^{\text{GRPO}} = \frac{R_i - \text{mean}_{j \in g}(R_j)}{\text{std}(R)}, \quad A_{i,s}^{\text{GRPO}} = A_i^{\text{GRPO}} \quad \forall s. \quad (2)$$

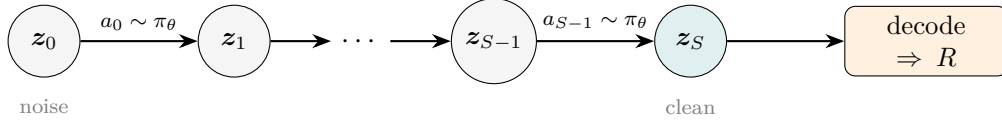


Figure 1: Denoising rollout as a finite-horizon MDP. The policy is the per-step SDE Gaussian transition; the reward is terminal (on the decoded final latent z_S). The critic estimates $V_\phi(z_s, t_s)$ at every step.

PPO. Critic-based PPO instead learns V_ϕ and forms per-step advantages by GAE (Section 4.2). The baseline is now state-aware ($V_\phi(z_s, t_s)$ rather than a group mean), and credit is distributed per step. Table 1 summarizes the structural difference; everything else (rollout, per-step log-probabilities, SDE dynamics, optional KL-to-reference) is shared.

	GRPO	Critic-based PPO
Advantage	one scalar / sample, broadcast	per-step A_s via GAE
Baseline	per-group reward mean	learned $V_\phi(z_s, t_s)$
Extra parameters	none	lightweight value critic + its optimizer
Reward normalization	group-relative (mean/std)	none on R ; advantages whitened
Per-epoch hook order	<code>sample()</code> $\rightarrow$ <code>prepare_feedback()</code> $\rightarrow$ <code>optimize()</code>	

Table 1: GRPO vs. critic-based PPO. Only the credit-assignment / baseline differs.

4 Algorithm and Data Flow

Figure 2 shows the per-epoch data flow. The trainer reuses `generate_samples` for rollout, then computes *old* values with the current critic (no grad), runs GAE, stores per-step targets on each sample, and finally optimizes.

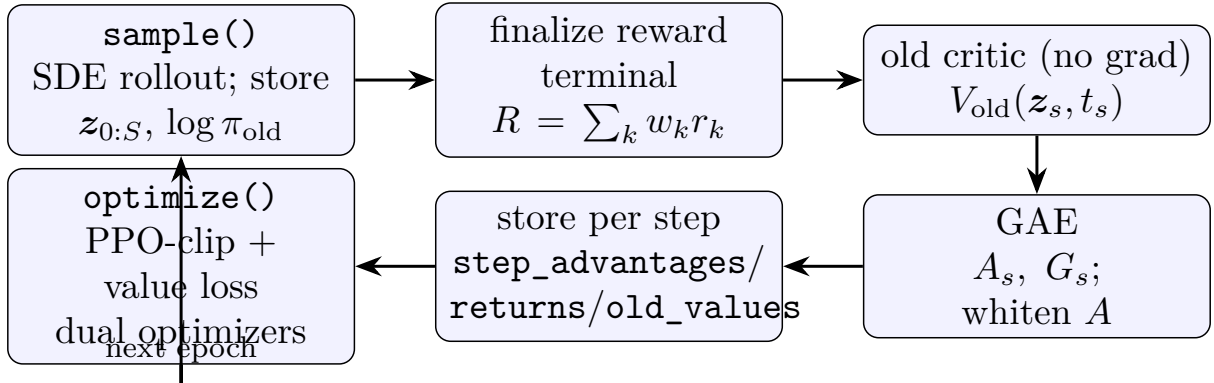


Figure 2: Per-epoch pipeline of the PPO trainer. `prepare_feedback()` covers the reward $\rightarrow$ old-value $\rightarrow$ GAE $\rightarrow$ store stages; `optimize()` does the gradient update.

4.1 Value critic

The critic is an independent, resolution-agnostic network (`trainers/ppo/critic.py`). Any latent layout is folded to (B, seq, C) by moving the channel axis last (`latents_to_tokens` using `resolve_latent_axes`); tokens are encoded, aggregated over the sequence by *learned-query*

multi-head attention pooling (PMA-style, $O(\text{seq})$), concatenated with a sinusoidal timestep embedding, and mapped to a scalar (Figure 3). It is built eagerly in `_initialization()` from the channel count returned by `BaseAdapter.compute_actual_latent_shape`, so it needs no rollout to size and its checkpoint stays valid across resolutions.

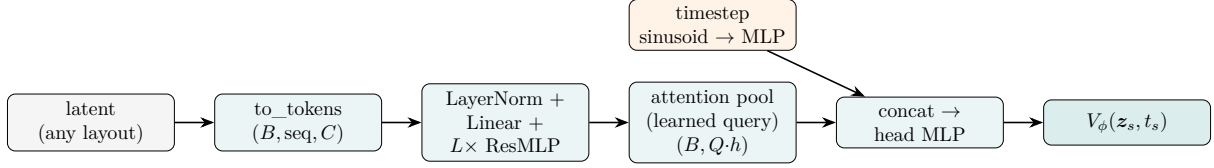


Figure 3: Value-critic architecture. Attention pooling keeps the network resolution/frame agnostic (it only depends on the channel count C) while preserving more information than mean/std pooling.

4.2 Generalized Advantage Estimation

With V_{old} evaluated at each ordered SDE step, the TD residual and GAE advantage are

$$\delta_s = r_s + \gamma V(\mathbf{z}_{s+1}) - V(\mathbf{z}_s), \quad V(\mathbf{z}_S) \equiv 0, \quad (3)$$

$$A_s = \sum_{l=0}^{S-1-s} (\gamma\lambda)^l \delta_{s+l}, \quad G_s = A_s + V(\mathbf{z}_s), \quad (4)$$

computed by the backward recursion $A_s = \delta_s + \gamma\lambda A_{s+1}$ (`compute_gae`). G_s is the value-regression target. With a terminal-only reward and $\gamma = \lambda = 1$ this collapses to $A_s = R - V(\mathbf{z}_s)$ and $G_s = R$ for all s (the critic-baselines analogue of GRPO); $\lambda < 1$ trades variance for bias. The example config uses a *full, contiguous* SDE trajectory so the bootstrap $V(\mathbf{z}_{s+1})$ is the value of the actual next stored latent. Advantages are optionally whitened across all (sample, step) pairs and ranks via a single all-reduce (`whiten` $\rightarrow$ `utils.dist.global_tensor_stats`).

4.3 PPO objective

Let $\rho_s = \exp(\log \pi_\theta(a_s | \mathbf{z}_s) - \log \pi_{\text{old}}(a_s | \mathbf{z}_s))$ be the per-step importance ratio and $\hat{A}_s$ the (clamped, whitened) advantage. The clipped objective is

$$\mathcal{L}^{\text{pol}} = -\mathbb{E}_s[\min(\rho_s \hat{A}_s, \text{clip}(\rho_s, 1+c_-, 1+c_+) \hat{A}_s)], \quad (5)$$

$$\mathcal{L}^{\text{val}} = \frac{1}{2} \mathbb{E}_s[\max((V_\phi - G_s)^2, (V_\phi^{\text{clip}} - G_s)^2)], \quad V_\phi^{\text{clip}} = V_{\text{old}} + \text{clip}(V_\phi - V_{\text{old}}, \pm \epsilon_v), \quad (6)$$

$$\mathcal{L} = \mathcal{L}^{\text{pol}} + c_v \mathcal{L}^{\text{val}} + \beta \text{KL}(\pi_\theta \| \pi_{\text{ref}}). \quad (7)$$

As in Flow-GRPO, the ratio clip range (c_-, c_+) is intentionally tiny (default $\pm 10^{-4}$) because $\log \pi$ is a per-dimension average over a high-dimensional Gaussian. The KL-to-reference term ($\beta = \text{kl_beta}$) is optional and off by default. Policy and critic use *separate* backward passes (each only traverses its own sub-graph) and each steps its own AdamW optimizer (`optimizer`, `critic_optimizer`); this independence is what lets their update frequencies be decoupled (§4.5).

Critic warmup as an up-front bootstrap. The initial critic warmup and the periodic re-warmup (§4.6) share a single critic-only path. Before the epoch loop, `BOOTSTRAPCRITIC` resamples fresh rollouts and fits *only* the critic until it has taken $\sim \text{critic_warmup_steps}$ optimizer steps, so a value baseline is established before any policy update (the “value pretraining” idea of [1]). Unlike a warmup woven into the first epochs, this bootstrap runs once up front and does *not* freeze them: once the loop starts the policy updates every epoch.

4.4 Pseudocode

The algorithm has two committed stages; both are kept for reference. Algorithm 1 is the original formulation (commit 4bdec9d): a single combined loss $\mathcal{L} = c_v \mathcal{L}^{\text{val}} + \mathcal{L}^{\text{pol}}(+\beta \text{KL})$, one `accelerator.backward`, a shared `accumulate` window, both optimizers stepping together at the sync boundary, and the warmup woven into the first epochs. Algorithm 2 is the current formulation (commit 4145161): separate per-network backward passes, decoupled update intervals (§4.5), and the warmup unified with periodic critic re-warmup (§4.6).

Algorithm 1 Critic-based PPO — v1: combined loss, shared `accumulate` (commit 4bdec9d)

```

1:  $\mathcal{D} \leftarrow \text{SAMPLE}()$ ;  $\{R_i\} \leftarrow \text{REWARD}(\mathcal{D})$ ;  $\mathcal{S} \leftarrow \text{sorted}(\text{train SDE steps})$ 
2:  $V_{\text{old}} \leftarrow V_{\phi}(\mathcal{D})$  (no grad);  $A, G \leftarrow \text{GAE}$ ;  $A \leftarrow \text{WHITEN}(A)$ ; store on samples
3: for inner epoch = 1, ...,  $N$  do
4:   for micro-batch  $b$  do ▷ with accumulate(model_bundle, critic)
5:     for  $s \in \mathcal{S}$  do
6:        $V \leftarrow V_{\phi}(z_s^b, t_s)$ ;  $\mathcal{L} \leftarrow c_v \mathcal{L}^{\text{val}}$ 
7:       if  $r \geq \text{critic\_warmup\_steps}$  then
8:          $\mathcal{L}^{\text{pol}} \leftarrow \text{Eq. (5)}$ ;  $\mathcal{L} \leftarrow \mathcal{L} + \mathcal{L}^{\text{pol}}(+\beta \text{KL})$  ▷ policy forward
9:       else
10:        policy frozen; forward skipped
11:      end if
12:      BACKWARD( $\mathcal{L}$ ) ▷ single combined backward; accelerate  $\div gas$ 
13:      if sync_gradients at round end then
14:        clip + step critic_optimizer (and optimizer unless warmup); zero both
15:      end if
16:    end for
17:  end for
18: end for

```

Algorithm 2 Critic-based PPO — current: separate backward, decoupled frequency + re-warmup (commit 4145161)

```

1: once, before training: BOOTSTRAPCRITIC (§4.6); every critic_rewarmup_interval
   rounds: REWARMUPCRITIC
2:  $\mathcal{D} \leftarrow \text{SAMPLE}()$  ▷ SDE rollout; store  $\mathbf{z}_{0:S}$ ,  $\log \pi_{\text{old}}$  at SDE steps
3:  $\{R_i\} \leftarrow \text{REWARD}(\mathcal{D})$  ▷ terminal weighted-sum score per sample
4:  $\mathcal{S} \leftarrow \text{sorted}(\text{train SDE steps})$ 
5: for micro-batch  $b$  in  $\mathcal{D}$  do ▷ no grad
6:   for  $s \in \mathcal{S}$  do
7:      $V_{\text{old}}[b, s] \leftarrow V_{\phi}(\mathbf{z}_s^b, t_s)$ 
8:   end for
9: end for
10:  $A, G \leftarrow \text{GAE}(V_{\text{old}}, R, \gamma, \lambda)$ ;  $A \leftarrow \text{WHITEN}(A)$  ▷ global, cross-rank
11: store  $A[\cdot, s], G[\cdot, s], V_{\text{old}}[\cdot, s]$  on each sample
12: for inner epoch = 1, ...,  $N$  do
13:   for micro-batch  $b$  do ▷ round  $r$ ;  $gas$  micro-steps/round;  $c_i, p_i$  = update intervals
14:     for  $s \in \mathcal{S}$  do
15:       if update_critic_in_optimize then ▷ mode A; mode B trains critic via bursts
16:          $V \leftarrow V_{\phi}(\mathbf{z}_s^b, t_s)$ ;  $\text{BACKWARD}(c_v \mathcal{L}^{\text{val}}/c_i)$  ▷ no_sync off only at critic
        boundary
17:       end if
18:        $\mathcal{L}^{\text{pol}} \leftarrow \text{Eq. (5)}$ ;  $\text{BACKWARD}((\mathcal{L}^{\text{pol}} + \beta \text{KL})/p_i)$  ▷ policy every micro-step
19:       if round end and mode A and  $c_i \mid (r+1)$  then
20:         clip + step critic_optimizer; zero grad
21:       end if
22:       if round end and  $p_i \mid (r+1)$  then
23:         clip + step optimizer; zero grad
24:       end if
25:     end for
26:   end for
27: end for

```

Algorithm 3 GAE (terminal-only reward)

```

Require: values  $V \in \mathbb{R}^{B \times S}$ , terminal rewards  $R \in \mathbb{R}^B$ ,  $\gamma, \lambda$ 
1:  $A \leftarrow \mathbf{0}_{B \times S}$ ;  $A_{\text{next}} \leftarrow \mathbf{0}_B$ 
2: for  $s = S - 1, \dots, 0$  do
3:    $r \leftarrow R$  if  $s = S - 1$  else  $\mathbf{0}$ ;  $V_{\text{next}} \leftarrow \mathbf{0}$  if  $s = S - 1$  else  $V[:, s+1]$ 
4:    $\delta \leftarrow r + \gamma V_{\text{next}} - V[:, s]$ 
5:    $A_{\text{next}} \leftarrow \delta + \gamma \lambda A_{\text{next}}$ ;  $A[:, s] \leftarrow A_{\text{next}}$ 
6: end for
7: return  $A, A + V$ 

```

4.5 Decoupling critic and policy update frequency

Because the critic and policy use *separate* backward passes, their optimizer steps can run at different cadences. Let a *round* be one base gradient-accumulation window of *gas* micro-steps. Each network keeps its own window: with intervals $c_i = \text{critic_update_interval}$ and $p_i = \text{policy_update_interval}$ (in rounds), the critic accumulates c_i *gas* micro-steps before each step and the policy accumulates p_i *gas*. **Both networks still forward and backward on every micro-step**, so no data is dropped; a larger interval only enlarges the effective batch

and lowers the step frequency of that network. Setting $c_i=1, p_i=K$ makes the critic relatively $K\times$ faster; swapping them reverses the direction.

Gradient scaling. `accelerator.backward` already divides the loss by gas . To make the accumulated gradient the *mean* over a network’s full ($interval \cdot gas$)-micro-step window, each loss is additionally divided by its own interval before backward:

$$g = \frac{1}{c_i gas} \sum_{j=1}^{c_i gas} \nabla_{\phi}(c_v \mathcal{L}_j^{\text{val}}), \quad g = \frac{1}{p_i gas} \sum_{j=1}^{p_i gas} \nabla_{\theta}(\mathcal{L}_j^{\text{pol}} + \beta \text{KL}_j), \quad (8)$$

so the effective learning rate is invariant to the interval. Gradient synchronization is deferred with per-network `no_sync` until each window closes; window boundaries are tracked by a self-maintained micro-step counter, not Accelerate’s shared `sync_gradients`. Since the prepared optimizers’ `step/zero_grad` are themselves gated on that flag, we pin `sync_gradients=True` at `optimize/burst` entry so they always fire, and (manual-`gas` only) assert the per-`optimize` micro-step count is a multiple of `gas` so a window never straddles two epochs’ rollouts. This is the two-timescale idea of TD3 / DMD2 (update one network less often to stabilize the other), here realized as accumulation-window decoupling on a fixed on-policy buffer rather than as fresh-minibatch skipping.

Caveat (fixed targets within an epoch). The per-step advantages $\hat{A}_s$ and returns G_s are computed once in `PREPARE_FEEDBACK` using the *old* critic and are frozen for the whole `optimize` epoch (Eq. (5)). Hence stepping the critic more often does *not* sharpen the current epoch’s policy targets; its benefit is cross-iteration (a better baseline for the *next* epoch’s GAE). A genuinely “moving” target would require recomputing returns more frequently within the epoch, which is out of scope here.

4.6 Periodic critic re-warmup (resample)

Interval decoupling (§4.5) cannot make the critic update *more* often than the policy: the finest cadence is one step per round. To let the critic periodically “catch up” to a moving policy we add a *re-warmup burst*, the same critic-only operation as the initial bootstrap. Every `critic_rewarmup_interval` optimizer rounds (checked after `optimize`; 0 disables) the trainer runs `REWARMUPCRITIC`: set a fresh seed, `SAMPLE` new rollouts from the current policy, recompute returns with the current critic via `PREPARE_FEEDBACK`, and fit *only* the critic for `critic_rewarmup_inner_epochs` passes. A burst never forwards or steps the policy and never advances `self.step / self.epoch`, so the policy still updates every epoch; bursts are extra critic-only work inserted between epochs (one resampled rollout each).

Two regimes via `update_critic_in_optimize`. With the default `True` (mode A) the critic also trains jointly inside `optimize`; bursts are an additional boost. With `False` (mode B) the critic is frozen inside `optimize` and trained *only* by the bootstrap and the periodic bursts – a policy/critic *alternation*: the policy advances every epoch on a frozen value baseline, and the critic re-fits in bursts (validated to require `critic_rewarmup_interval > 0`). Because the resample draws from the *current* policy, a burst aligns the critic to the present distribution – the “moving target” the per-epoch frozen returns (§4.5 caveat) cannot. The cost is one extra rollout per burst, so bursts are typically infrequent (e.g. every 50 epochs).

5 Theoretical Analysis

5.1 State-dependent baselines are unbiased and reduce variance

Proposition 1 (Baseline). *For any function $b(\mathbf{z}_s, t_s)$ of the state only,*

$$\mathbb{E}\left[\sum_s \nabla_\theta \log \pi_\theta(a_s \mid \mathbf{z}_s, t_s) (G_s - b(\mathbf{z}_s, t_s))\right] = \nabla_\theta J(\theta),$$

i.e. subtracting a state-dependent baseline leaves the policy gradient unbiased. The per-step gradient variance is minimized by $b \approx \mathbb{E}[G_s \mid \mathbf{z}_s, t_s] = V^\pi(\mathbf{z}_s, t_s)$.

Proof sketch. For a fixed state, the baseline contributes zero in expectation:

$$\mathbb{E}_{a_s \sim \pi}[\nabla_\theta \log \pi_\theta(a_s \mid \cdot) b(\mathbf{z}_s, t_s)] = b(\mathbf{z}_s, t_s) \nabla_\theta \int \pi_\theta da_s = b(\mathbf{z}_s, t_s) \nabla_\theta 1 = 0.$$

Variance is a quadratic in b minimized near the conditional mean of the return, which is the value function. $\square$

Proposition 1 justifies replacing GRPO’s group-mean baseline by the learned V_ϕ : a state-conditioned baseline is the variance-optimal choice and is strictly more informative than a per-group constant.

5.2 GAE bias–variance

$A_s(\gamma, \lambda) = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{s+l}$ interpolates between the one-step TD estimator ($\lambda=0$: low variance, biased by critic error) and the Monte-Carlo return minus baseline ($\lambda=1$: unbiased w.r.t. the true value, high variance). With a learned, imperfect V_ϕ , an intermediate λ (default 0.95) typically minimizes mean-squared advantage error. Because the reward is terminal, larger λ propagates the single terminal signal further back along the trajectory, while V_ϕ shapes the intermediate credit.

5.3 Trust-region surrogate

Equation (5) is the PPO clipped surrogate: it lower-bounds the policy improvement while preventing destructive updates when ρ_s drifts from 1. The on-policy ratio $\rho_s = 1$ at the first inner epoch (rollout policy = optimization policy), and clipping bounds the step as inner epochs reuse the rollout data.

5.4 Train–inference consistency

The actor’s $\log \pi_{\text{old}}$ is produced by the *same* adapter forward and scheduler step used at training time, stored during rollout; the optimize-time forward recomputes $\log \pi_\theta$ identically up to the parameter update. This keeps ρ_s an honest importance ratio—an invariant Flow-Factory enforces across all coupled algorithms.

6 Comparison with *Explicit Critic Guidance* (arXiv:2605.27736)

Liang et al. [1] propose a *state-aligned latent actor–critic* for diffusion post-training in which **the diffusion model serves as its own timestep-conditioned value function**, predicting values directly on noisy latent states. This enables trajectory-level PPO, stabilized by simple conditioning and *value pretraining*, and—because the value head shares the backbone—the learned critic can be reused for *inference-time steering*. They further train with multiple complementary rewards to mitigate reward hacking, and report gains over group-relative RL (GRPO) and actor–critic baselines on UNet- and DiT-based backbones.

Flow-Factory’s PPO shares the same core thesis—*state-aligned value estimation on noisy latents with per-step PPO credit*—but makes the opposite engineering choice for the critic body. The paper’s “diffusion-model-as-value-function” is exactly the **backbone+value-head critic** that Flow-Factory evaluated and deliberately *deferred* in favor of an independent, lightweight, model-agnostic critic. Figure 4 and Table 2 contrast the two.

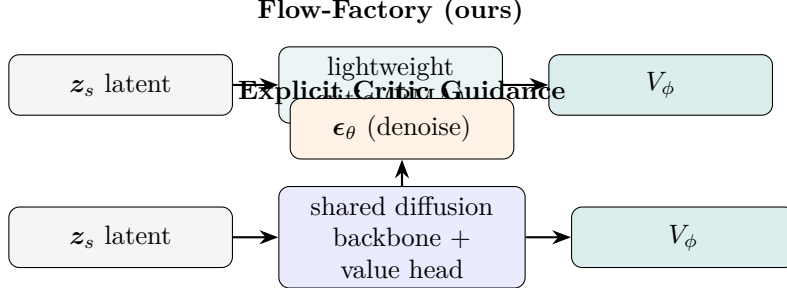


Figure 4: Critic body: independent lightweight network (ours) vs. the diffusion backbone reused as its own value function [1].

Aspect	Flow-Factory PPO (ours)	Explicit Critic Guidance [1]
Critic body	independent lightweight net; attention-pool on latent tokens	diffusion backbone is its own value function (shared weights + value head)
State alignment	$V_\phi(z_s, t_s)$ on noisy latent (state-aligned)	value predicted on noisy latent states (state-aligned)
Credit assignment	GAE per step	trajectory-level PPO, fine-grained credit
Value init	critic warmup (critic-only steps)	value pretraining strategies
Inference-time steering	not supported (follow-up)	critic reused for test-time steering (extra gains)
Multi-reward	terminal weighted sum; source-aware deferred	joint multi-reward to reduce reward hacking
Backbones	model-agnostic (conv / packed / video)	UNet and DiT
Cost	cheap: small critic, $O(\text{seq})$ pooling	heavier: a backbone forward per value query

Table 2: Side-by-side comparison. Both are state-aligned actor–critic PPO for diffusion; the main axis of difference is the critic body and what it unlocks (cheap + model-agnostic vs. stronger value capacity + inference-time steering).

Discussion. The paper is strong evidence that the backbone-critic direction pays off, especially because a shared backbone (i) gives the value head the policy’s rich representation, (ii) conditions on the full prompt/condition for free, and (iii) enables test-time steering. Flow-Factory’s v1 trades some of that value-estimation capacity for a critic that is essentially free in memory/compute and works across every adapter without per-architecture surgery. The two designs are complementary: the lightweight critic is the safe default and reference baseline, while the backbone+value-head critic (and its inference-time steering and source-aware multi-reward) are the natural next upgrades, for which this PPO trainer already provides the GAE / dual-optimizer / checkpoint scaffolding.

7 Implementation Notes and Configuration

- **Files.** `trainers/ppo/{critic,gae,trainer}.py`, `hparams/training_args/ppo.py`, and `models/latent_geometry.py` (`compute_actual_latent_shape / latent_shape`). Example: `examples/ppo/lora/sd3_5/default.yaml`.
- **Paradigm.** Coupled paradigm: requires SDE dynamics (Flow-SDE, Dance-SDE, or CPS). Inherits `BaseTrainer`.
- **Distributed.** v1 targets DDP (the critic uses a second optimizer); DeepSpeed/FSDP support for the second optimizer is a follow-up. The dual-optimizer manual-accumulation path is verified for bf16; fp16 (a shared `GradScaler` across two optimizers) is a follow-up.
- **Update-frequency decoupling.** `critic_update_interval / policy_update_interval` (in optimizer rounds) set per-network accumulation windows; see §4.5.
- **GAE assumption.** Configure a full, contiguous SDE trajectory (`scheduler.sde_steps: null, num_sde_steps = num_inference_steps - 1`) so the per-step bootstrap is exact.

Hyperparameter	Default	Meaning
<code>critic_learning_rate</code>	1e-4	critic AdamW LR
<code>vf_coef</code>	0.5	value-loss weight c_v
<code>gae_gamma / gae_lambda</code>	1.0 / 0.95	γ / λ
<code>value_clip_range</code>	0.2	value clip ϵ_v (None to disable)
<code>clip_range</code>	$\pm 10^{-4}$	policy ratio clip (c_- , c_+)
<code>normalize_advantage</code>	true	global advantage whitening
<code>critic_warmup_steps</code>	30	critic-only warmup (policy frozen)
<code>critic_update_interval</code>	1	step critic every N rounds
<code>policy_update_interval</code>	1	step policy every N rounds
<code>critic_hidden_dim / _num_layers</code>	256 / 3	critic width / depth
<code>critic_attn_heads / _num_query_tokens</code>	8 / 1	attention-pool config
<code>kl_beta / kl_type</code>	0 / x-based	optional KL-to-reference

Table 3: Key `PPOTrainingArguments` fields.

8 Limitations and Future Work

- **Backbone+value-head critic** (as in [1]): stronger value capacity and free conditioning, at $\sim 2\times$ memory/compute; needs per-architecture feature taps. Evaluated and deferred.
- **Inference-time steering** with the learned critic (requires the shared backbone).
- **Text / condition conditioning** of the critic (pooled or full prompt embeddings) and **source-aware multi-reward** aggregation.
- **DeepSpeed/FSDP** support for the critic’s second optimizer.

References

- [1] Zhengyang Liang, Qihang Zhang, Ceyuan Yang. Explicit Critic Guidance for Aligning Diffusion Models. arXiv:2605.27736, 2026. <https://arxiv.org/abs/2605.27736>